



第三章 程序的转换及机器级表示

主要内容



华中科技大学

- ◆ 3.1 程序转换概述
- ◆ 3.2 IA-32指令系统概述
- ◆ 3.3 IA-32常用指令类型及其操作
- ◆ 3.4 C语言的机器级表示
- ◆ 3.5 复杂数据类型的分配和访问
- ◆ 3.6 越界访问和缓冲区攻击





一、地址和指针

1.汇编语言中的地址和指针

- 地址是对主存中的每个字节单元，指定一个编号
- 变量的内容，使用时没有类型限制
- 汇编语言中没有指针类型
- 一个数值作为地址使用时，通过寄存器进行间接寻址，找到目标操作数。



2.C语言中的地址和指针

- C语言中，变量是在内存中分配的单元
- 每个变量都有一个地址，是变量的内存单元的第一个字节的编号
- 使用运算符 & 获取变量的地址
- 变量的地址，是一个无符号整数型的常量
- C语言中的指针，是一种变量类型，其内容做为地址使用
- 使用 `type *`，定义指针变量
- 指针变量，其内容做为间接寻址使用。需要将变量值送到寄存器中进行间接寻址
- 指针类型定义中的`type`，是间接寻址后的对象的数据类型。只是使用建议，没有强制作用





二、数组的分配和访问

数组是在内存中分配的一段连续区域，并使用数组下标访问数组元素。

数组的分配是编译程序时进行的。



1.数组的定义和存储

数组的存储类型:

- 静态存储——static
- 外部存储——extern
- 自动存储——auto

auto型存储分配在堆栈段中，其他存储类型分配在静态数据区（数据段中）



1.数组的定义和存储

例:

数组定义	数组名	数组元素类型	数组元素大小 (B)	数组大小 (B)	起始地址	元素 i 的地址
char S[10]	S	char	1	10	&S[0]	&S[0]+i
char * SA[10]	SA	char *	4	40	&SA[0]	&SA[0]+4*i
double D[10]	D	double	8	80	&D[0]	&D[0]+8*i
double * DA[10]	DA	double *	4	40	&DA[0]	&DA[0]+4*i



2.数组的访问

例:

```
#include <stdio.h>
int main()
{
    int    a[10];
    char   b[50] = "1234567";
    for (int i = 0; i < 10; i++)
        a[i] = i * 3;
    for (int j = 0; j < 15; j++)
        putchar(b[j]);
    return 0;
}
```




2.数组的访问

```
5      int    a[10];
6      char    b[50] = "1234567";
=> 0x565561d5 <+40>:    movl    $0x34333231, -0x3e(%ebp)
    0x565561dc <+47>:    movl    $0x373635, -0x3a(%ebp)
    0x565561e3 <+54>:    movl    $0x0, -0x36(%ebp)
    0x565561ea <+61>:    movl    $0x0, -0x32(%ebp)
    0x565561f1 <+68>:    movl    $0x0, -0x2e(%ebp)
    0x565561f8 <+75>:    movl    $0x0, -0x2a(%ebp)
    0x565561ff <+82>:    movl    $0x0, -0x26(%ebp)
    0x56556206 <+89>:    movl    $0x0, -0x22(%ebp)
    0x5655620d <+96>:    movl    $0x0, -0x1e(%ebp)
    0x56556214 <+103>:   movl    $0x0, -0x1a(%ebp)
    0x5655621b <+110>:   movl    $0x0, -0x16(%ebp)
    0x56556222 <+117>:   movl    $0x0, -0x12(%ebp)
    0x56556229 <+124>:   movw    $0x0, -0xe(%ebp)
```



2.数组的访问

```
8          for (int i = 0; i < 10; i++)
0x5655622f <+130>:  movl    $0x0, -0x70(%ebp)
0x56556236 <+137>:  jmp     0x5655624c <main+159>
0x56556248 <+155>:  addl    $0x1, -0x70(%ebp)
0x5655624c <+159>:  cmpl    $0x9, -0x70(%ebp)
0x56556250 <+163>:  jle     0x56556238 <main+139>

9          a[i] = i * 3;
0x56556238 <+139>:  mov     -0x70(%ebp), %edx
0x5655623b <+142>:  mov     %edx, %eax
0x5655623d <+144>:  add     %eax, %eax
0x5655623f <+146>:  add     %eax, %edx
0x56556241 <+148>:  mov     -0x70(%ebp), %eax
0x56556244 <+151>:  mov     %edx, -0x68(%ebp, %eax, 4)
```



2.数组的访问

```
10
11      for (int j = 0; j < 15; j++)
0x56556252 <+165>:  movl    $0x0,-0x6c(%ebp)
0x56556259 <+172>:  jmp     0x56556279 <main+204>
0x56556275 <+200>:  addl    $0x1,-0x6c(%ebp)
0x56556279 <+204>:  cmpl    $0xe,-0x6c(%ebp)
0x5655627d <+208>:  jle     0x5655625b <main+174>

12      putchar(b[j]);
0x5655625b <+174>:  lea     -0x3e(%ebp),%edx
0x5655625e <+177>:  mov     -0x6c(%ebp),%eax
0x56556261 <+180>:  add     %edx,%eax
0x56556263 <+182>:  movzbl  (%eax),%eax
0x56556266 <+185>:  movsbl  %al,%eax
0x56556269 <+188>:  sub     $0xc,%esp
0x5655626c <+191>:  push    %eax
--Type <RET> for more, q to quit, c to continue without paging--
0x5655626d <+192>:  call    0x56556060 <putchar@plt>
```





2.数组的访问

一维数组数组元素的访问:

- 采用变址寻址
- 数组名作为位移量
- 数组下标放到变址寄存器中

3.数组和指针



华中科技大学

**在指针变量目标数据类型与数组类型相同的前提下
， 指针变量可以指向数组或数组中任意元素。**





3.数组和指针

例：

```
#include <stdio.h>
int main()
{
    int a[10];
    int *ptr1 = &a[0];
    int *ptr2 = &a[1];
    int *ptr0 = a;
    return 0;
}
```



3. 数组和指针

```
5      int a[10];
6      int *ptr1 = &a[0];
=> 0x565561bc <+31>:  lea    0x14(%esp),%eax
    0x565561c0 <+35>:  mov     %eax,0x8(%esp)

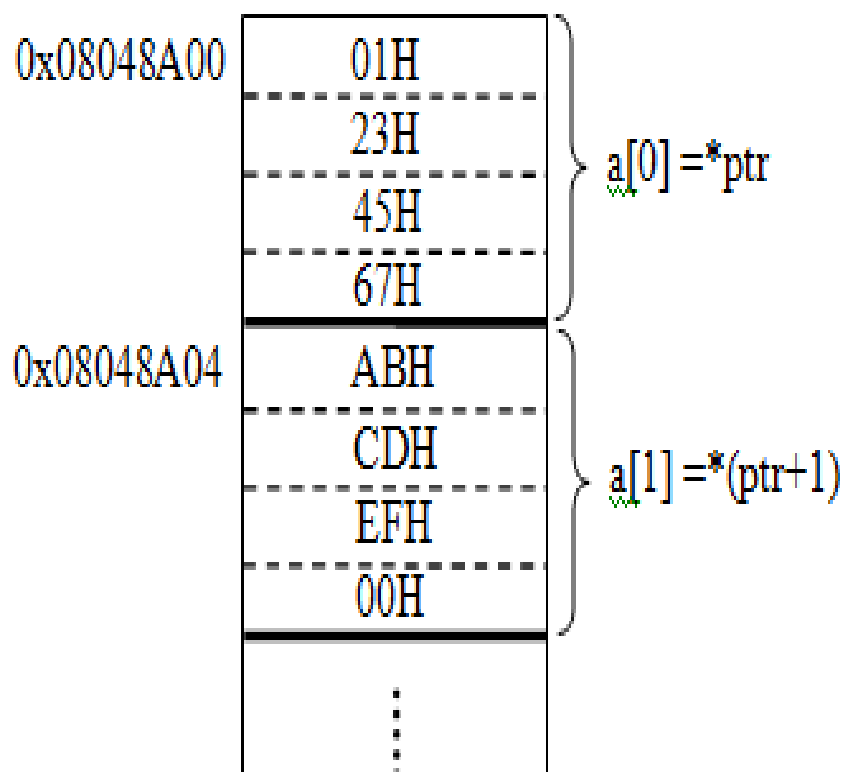
7      int *ptr2 = &a[1];
    0x565561c4 <+39>:  lea    0x14(%esp),%eax
    0x565561c8 <+43>:  add     $0x4,%eax
    0x565561cb <+46>:  mov     %eax,0xc(%esp)

8      int *ptr0 = a;
--Type <RET> for more, q to quit, c to continue without paging--
    0x565561cf <+50>:  lea    0x14(%esp),%eax
    0x565561d3 <+54>:  mov     %eax,0x10(%esp)

9      return 0;
    0x565561d7 <+58>:  mov     $0x0,%eax
```

3. 数组和指针

其中，指针ptr1指向数组a的第0个元素a[0]，指针ptr2指向数据a的第1个元素a[1]。指针ptr0和ptr1等价，也指向a[0]。即a和&a[0]等价的，都是数组a首个元素的地址。





3.数组和指针

例:

序号	表达式	类型	值的计算方式	汇编代码
1	A	int *	问题: 假定数组A的首址SA在ECX中, i在EDX中, 表达式结果在EAX中, 各表达式的计算方式以及汇编代码各是什么?	
2	A[0]	int		
3	A[i]	int		
4	&A[3]	int *		
5	&A[i]-A	int		
6	*(A+i)	int		
7	*(&A[0]+i-1)	int		
8	A+i	int *		



3.数组和指针

序号	表达式	类型	值的计算方式	汇编代码
1	A	int *	SA	leal (%ecx), %eax
2	A[0]	int	M[SA]	movl (%ecx), %eax
3	A[i]	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
4	&A[3]	int *	SA+12	leal 12(%ecx), %eax
5	&A[i]-A	int	$(SA+4*i-SA)/4=i$	movl %edx, %eax
6	*(A+i)	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
7	*(&A[0]+i-1)	int	M[SA+4*i-4]	movl -4(%ecx, edx, 4), %eax
8	A+i	int *	SA+4*i	leal (%ecx, %edx, 4), %eax

2、3、6和7对应汇编指令都需访存，指令中源操作数的寻址方式分别是“基址”、“基址加比例变址”、“基址加比例变址”和“基址加比例变址加位移”的方式，因为数组元素的类型为int型，故比例因子为4。





4. 指针数组和 多维数组

a) 指针数组

数组元素是指针变量的数组，称为指针数组。

例：

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char*  sa[5] = {"hello", "abc", "123", "yes", "no"};
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%s\n", sa[i]);
```

```
    }
```

```
    return 0;
```

```
}
```





a) 指针数组

```
5      char*   sa[5] = {"hello", "abc", "123", "yes", "no"};
```

```
=> 0x565561d5 <+40>:  lea    -0x1fcc(%ebx),%eax
```

```
0x565561db <+46>:  mov    %eax,-0x20(%ebp)
```

```
0x565561de <+49>:  lea    -0x1fc6(%ebx),%eax
```

```
0x565561e4 <+55>:  mov    %eax,-0x1c(%ebp)
```

```
0x565561e7 <+58>:  lea    -0x1fc2(%ebx),%eax
```

```
0x565561ed <+64>:  mov    %eax,-0x18(%ebp)
```

```
--Type <RET> for more, q to quit, c to continue without paging--
```

```
0x565561f0 <+67>:  lea    -0x1fbe(%ebx),%eax
```

```
0x565561f6 <+73>:  mov    %eax,-0x14(%ebp)
```

```
0x565561f9 <+76>:  lea    -0x1fba(%ebx),%eax
```

```
0x565561ff <+82>:  mov    %eax,-0x10(%ebp)
```

```
6      for (int i = 0; i < 5; i++)
```

```
0x56556202 <+85>:  movl   $0x0,-0x24(%ebp)
```

```
0x56556209 <+92>:  jmp    0x56556222 <main+117>
```

```
0x5655621e <+113>:  addl   $0x1,-0x24(%ebp)
```

```
0x56556222 <+117>:  cmpl   $0x4,-0x24(%ebp)
```

```
0x56556226 <+121>:  jle    0x5655620b <main+94>
```



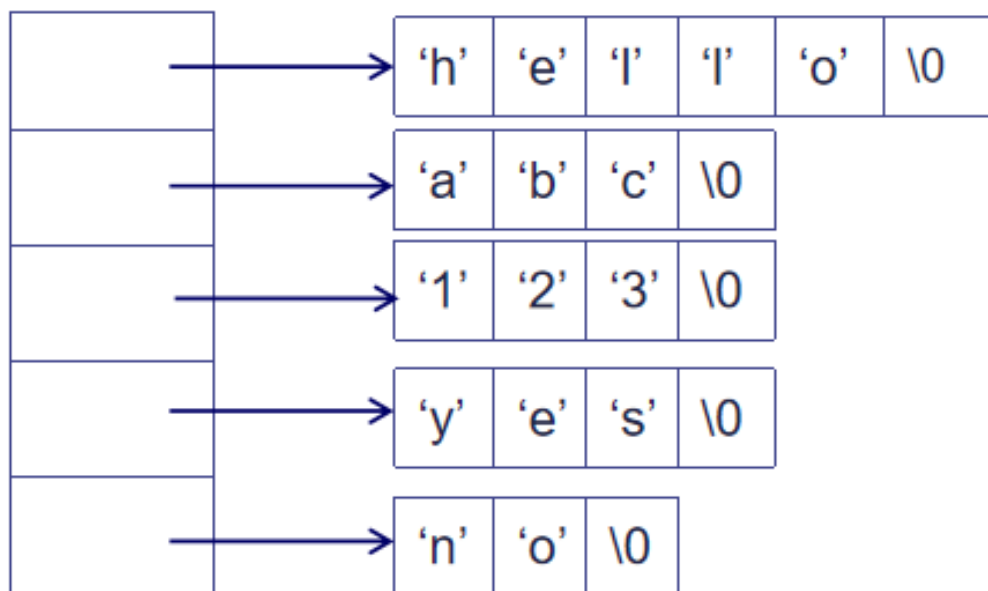


a) 指针数组

```
7      {  
8          printf("%s\n", sa[i]);  
0x5655620b <+94>:    mov     -0x24(%ebp),%eax  
0x5655620e <+97>:    mov     -0x20(%ebp,%eax,4),%eax  
0x56556212 <+101>:   sub     $0xc,%esp  
0x56556215 <+104>:   push    %eax  
0x56556216 <+105>:   call   0x56556060 <puts@plt>  
0x5655621b <+110>:   add     $0x10,%esp  
9      }
```

a) 指针数组

- sa是一个数组
- 每个数组元素是一个指针
- 指针所指对象的类型是char



b) 多维数组



华中科技大学

有多个维度的数组称为多维数组，多维数组的元素有多个下标。

例：

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char a[3][5];
```

```
    char *p00 = &(a[0][0]);
```

```
    char *p01 = &(a[0][1]);
```

```
    char *p02 = &(a[0][2]);
```

```
    char *p10 = &(a[1][0]);
```

```
    char *p11 = &(a[1][1]);
```

```
    char *p20 = &(a[2][0]);
```

```
    char *p24 = &(a[2][4]);
```

```
    return 0;
```

```
}
```





b) 多维数组

```
5      char a[3][5];
6      char *p00 = &(a[0][0]);
=> 0x565561bc <+31>:  lea    0x1d(%esp),%eax
    0x565561c0 <+35>:  mov    %eax, (%esp)

7      char *p01 = &(a[0][1]);
    0x565561c3 <+38>:  lea    0x1d(%esp),%eax
    0x565561c7 <+42>:  add    $0x1,%eax
    0x565561ca <+45>:  mov    %eax, 0x4(%esp)

8      char *p02 = &(a[0][2]);
--Type <RET> for more, q to quit, c to continue without paging--
    0x565561ce <+49>:  lea    0x1d(%esp),%eax
    0x565561d2 <+53>:  add    $0x2,%eax
    0x565561d5 <+56>:  mov    %eax, 0x8(%esp)

9      char *p10 = &(a[1][0]);
    0x565561d9 <+60>:  lea    0x1d(%esp),%eax
    0x565561dd <+64>:  add    $0x5,%eax
    0x565561e0 <+67>:  mov    %eax, 0xc(%esp)
```



b) 多维数组

```
10      char *p11 = &(a[1][1]);  
0x565561e4 <+71>:  lea    0x1d(%esp),%eax  
0x565561e8 <+75>:  add    $0x6,%eax  
0x565561eb <+78>:  mov    %eax,0x10(%esp)  
  
11      char *p20 = &(a[2][0]);  
0x565561ef <+82>:  lea    0x1d(%esp),%eax  
0x565561f3 <+86>:  add    $0xa,%eax  
0x565561f6 <+89>:  mov    %eax,0x14(%esp)  
  
12      char *p24 = &(a[2][4]);  
0x565561fa <+93>:  lea    0x1d(%esp),%eax  
0x565561fe <+97>:  add    $0xe,%eax  
0x56556201 <+100>: mov    %eax,0x18(%esp)
```

- 多维数组在内存中实际分配的是一段一维的空间
- 在内存中按行优先的方式存放数组元素



三、结构体数据的分配和访问

1. 结构体的定义和存储

C语言的结构体将不同类型的数据依次存放在一段连续的存储区中。指向结构的指针，就是其第一个字节的地址。

结构体的存储类型：

- 静态存储——static
- 外部存储——extern
- 自动存储——auto

auto型存储分配在堆栈段中，其他存储类型分配在静态数据区（数据段中）。





1. 结构体的定义和存储

结构体成员首址，采用“变址寻址”方式确定：

结构体首地址 + 成员的偏移量

2. 结构体数据作为子程序参数和返回值



华中科技大学

当结构体变量需要作为一个函数的形参时，形参和调用函数中的实参应具有相同结构。

若采用按值传递，则结构成员都要复制到栈中参数区，这既增加时间开销又增加空间开销，且更新后的数据无法在调用过程使用。因此，结构体参数通常按地址传递，即在执行CALL指令前，仅需传递指向结构体的指针，而不需复制每个成员到栈中。



2. 结构体数据作为子程序参数和返回值



华中科技大学

例：

```
#include <stdio.h>
typedef struct
{
    int    id;
    char   name[10];
} stu_info;
void reg_stu_v(stu_info s)
{
    s.id = 101;
} void reg_stu_a(stu_info* s)
{
    s->id = 201;
}
```

```
int main()
{
    stu_info stu = {-1, "jack"};
    reg_stu_v(stu);
    printf("%d\n", stu.id);
    reg_stu_a(&stu);
    printf("%d\n", stu.id);
    return 0;
}
```





2. 结构体数据作为子程序参数和返回值

```
20      {  
    0x565561dd <+0>:      lea      0x4(%esp),%ecx  
    0x565561e1 <+4>:      and      $0xffffffff0,%esp  
    0x565561e4 <+7>:      push     -0x4(%ecx)  
    0x565561e7 <+10>:     push     %ebp  
    0x565561e8 <+11>:     mov      %esp,%ebp  
    0x565561ea <+13>:     push     %ebx  
    0x565561eb <+14>:     push     %ecx  
    0x565561ec <+15>:     sub      $0x20,%esp  
    0x565561ef <+18>:     call     0x565560b0 <__x86.get_pc_thunk.bx>  
    0x565561f4 <+23>:     add      $0x2de0,%ebx  
    0x565561fa <+29>:     mov      %gs:0x14,%eax  
    0x56556200 <+35>:     mov      %eax,-0xc(%ebp)  
    0x56556203 <+38>:     xor      %eax,%eax  
  
21      stu_info stu = {-1, "jack"};  
=> 0x56556205 <+40>:     movl     $0xffffffff,-0x1c(%ebp)  
    0x5655620c <+47>:     movl     $0x6b63616a,-0x18(%ebp)  
    0x56556213 <+54>:     movl     $0x0,-0x14(%ebp)  
    0x5655621a <+61>:     movw     $0x0,-0x10(%ebp)
```





2. 结构体数据作为子程序参数和返回值

```
22      reg_stu_v(stu);
--Type <RET> for more, q to quit, c to continue without paging--
0x56556220 <+67>:    push    -0x10(%ebp)
0x56556223 <+70>:    push    -0x14(%ebp)
0x56556226 <+73>:    push    -0x18(%ebp)
0x56556229 <+76>:    push    -0x1c(%ebp)
0x5655622c <+79>:    call   0x565561ad <reg_stu_v>
0x56556231 <+84>:    add     $0x10,%esp

23      printf("%d\n", stu.id);
0x56556234 <+87>:    mov     -0x1c(%ebp),%eax
0x56556237 <+90>:    sub     $0x8,%esp
0x5655623a <+93>:    push    %eax
0x5655623b <+94>:    lea     -0x1fcc(%ebx),%eax
0x56556241 <+100>:   push    %eax
0x56556242 <+101>:   call    0x56556050 <printf@plt>
0x56556247 <+106>:   add     $0x10,%esp
```





2. 结构体数据作为子程序参数和返回值

```
24      reg_stu_a(&stu);
0x5655624a <+109>:  sub    $0xc,%esp
0x5655624d <+112>:  lea    -0x1c(%ebp),%eax
0x56556250 <+115>:  push   %eax
0x56556251 <+116>:  call   0x565561c4 <reg_stu_a>
0x56556256 <+121>:  add    $0x10,%esp

--Type <RET> for more, q to quit, c to continue without paging--
25      printf("%d\n", stu.id);
0x56556259 <+124>:  mov    -0x1c(%ebp),%eax
0x5655625c <+127>:  sub    $0x8,%esp
0x5655625f <+130>:  push   %eax
0x56556260 <+131>:  lea    -0x1fcc(%ebx),%eax
0x56556266 <+137>:  push   %eax
0x56556267 <+138>:  call   0x56556050 <printf@plt>
0x5655626c <+143>:  add    $0x10,%esp

26      return 0;
0x5655626f <+146>:  mov    $0x0,%eax
```


2. 结构体数据作为子程序参数和返回值



华中科技大学

```
    }  
0x56556274 <+151>:  mov    -0xc(%ebp),%edx  
0x56556277 <+154>:  sub    %gs:0x14,%edx  
0x5655627e <+161>:  je     0x56556285 <main+168>  
0x56556280 <+163>:  call   0x565562a0 <__stack_chk_fail_local>  
0x56556285 <+168>:  lea    -0x8(%ebp),%esp  
0x56556288 <+171>:  pop    %ecx  
0x56556289 <+172>:  pop    %ebx  
0x5655628a <+173>:  pop    %ebp  
0x5655628b <+174>:  lea    -0x4(%ecx),%esp  
0x5655628e <+177>:  ret
```



2. 结构体数据作为子程序参数和返回值



华中科技大学

汇编语言中，子程序通常使用EAX返回一个整数作为返回值。C语言子程序除了可以返回一个整型返回值外，也可以返回一个结构体返回值。



2. 结构体数据作为子程序参数和返回值



华中科技大学

例：

```
17      stu_info stu = get_stu();  
=> 0x56556231 <+38>:    lea     -0x1c(%ebp),%eax  
      0x56556234 <+41>:    sub     $0xc,%esp  
      0x56556237 <+44>:    push    %eax  
      0x56556238 <+45>:    call    0x5655619d <get_stu>  
      0x5655623d <+50>:    add     $0xc,%esp
```





2. 结构体数据作为子程序参数和返回值

```
10      {  
    0x5655619d <+0>:      push    %ebp  
    0x5655619e <+1>:      mov     %esp,%ebp  
    0x565561a0 <+3>:      sub     $0x38,%esp  
    0x565561a3 <+6>:      call    0x5655625e <__x86.get_pc_thunk.ax>  
    0x565561a8 <+11>:     add     $0x2e30,%eax  
    0x565561ad <+16>:     mov     0x8(%ebp),%eax  
    0x565561b0 <+19>:     mov     %eax,-0x2c(%ebp)  
    0x565561b3 <+22>:     mov     %gs:0x14,%eax  
    0x565561b9 <+28>:     mov     %eax,-0xc(%ebp)  
    0x565561bc <+31>:     xor     %eax,%eax  
  
11      stu_info stu = {75, "Peter"};  
    0x565561be <+33>:     movl    $0x4b,-0x1c(%ebp)  
    0x565561c5 <+40>:     movl    $0x65746550,-0x18(%ebp)  
    0x565561cc <+47>:     movl    $0x72,-0x14(%ebp)  
    0x565561d3 <+54>:     movw    $0x0,-0x10(%ebp)
```

2. 结构体数据作为子程序参数和返回值



华中科技大学

```
12      return stu;
0x565561d9 <+60>:  mov     -0x2c(%ebp),%eax
0x565561dc <+63>:  mov     -0x1c(%ebp),%edx
0x565561df <+66>:  mov     %edx,(%eax)
0x565561e1 <+68>:  mov     -0x18(%ebp),%edx
0x565561e4 <+71>:  mov     %edx,0x4(%eax)
0x565561e7 <+74>:  mov     -0x14(%ebp),%edx
0x565561ea <+77>:  mov     %edx,0x8(%eax)
0x565561ed <+80>:  mov     -0x10(%ebp),%edx
0x565561f0 <+83>:  mov     %edx,0xc(%eax)
--Type <RET> for more, q to quit, c to continue without paging--

13      }
0x565561f3 <+86>:  mov     -0xc(%ebp),%eax
0x565561f6 <+89>:  sub     %gs:0x14,%eax
0x565561fd <+96>:  je      0x56556204 <get_stu+103>
0x565561ff <+98>:  call    0x56556270 <__stack_chk_fail_local>
0x56556204 <+103>:  mov     -0x2c(%ebp),%eax
0x56556207 <+106>:  leave
0x56556208 <+107>:  ret     $0x4
```



2. 结构体数据作为子程序参数和返回值



华中科技大学

由此可见，C语言中，主程序为返回结构体事先分配一段缓存，在主程序的当前堆栈中。

主程序将EAX作为指向返回结构体的指针，预先传递给子程序。

子程序返回后，主程序通过EAX作为指针，访问返回的结构体。



四、联合体数据的分配和访问



华中科技大学

C语言中的联合体，各成员共享存储空间，按最大长度成员所需空间大小为目标。

联合体各成员具有相同的起始地址。





四、联合体数据的分配和访问

例：

```
#include <stdio.h>
union uarea
{
    char   c_data;
    short  s_data;
    int    i_data;
    long   l_data;
};
```

```
int main()
{
    union uarea u;
    char* a1 = &(u.c_data);
    short* a2 = &(u.s_data);
    int* a3 = &(u.i_data);
    long* a4 = &(u.l_data);
    return 0;
}
```





四、联合体数据的分配和访问

```
13      union uarea u;  
14      char* a1 = &(u.c_data);  
=> 0x565561bc <+31>:   lea     0x8(%esp),%eax  
    0x565561c0 <+35>:   mov     %eax,0xc(%esp)  
  
15      short* a2 = &(u.s_data);  
    0x565561c4 <+39>:   lea     0x8(%esp),%eax  
    0x565561c8 <+43>:   mov     %eax,0x10(%esp)  
  
16      int* a3 = &(u.i_data);  
    0x565561cc <+47>:   lea     0x8(%esp),%eax  
    0x565561d0 <+51>:   mov     %eax,0x14(%esp)  
  
17      long* a4 = &(u.l_data);  
    0x565561d4 <+55>:   lea     0x8(%esp),%eax  
    0x565561d8 <+59>:   mov     %eax,0x18(%esp)
```



四、联合体数据的分配和访问

```
13      union uarea u;  
14      char* a1 = &(u.c_data);  
=> 0x565561bc <+31>:    lea     0x8(%esp),%eax  
    0x565561c0 <+35>:    mov     %eax,0xc(%esp)  
  
15      short* a2 = &(u.s_data);  
    0x565561c4 <+39>:    lea     0x8(%esp),%eax  
    0x565561c8 <+43>:    mov     %eax,0x10(%esp)  
  
16      int* a3 = &(u.i_data);  
    0x565561cc <+47>:    lea     0x8(%esp),%eax  
    0x565561d0 <+51>:    mov     %eax,0x14(%esp)  
  
17      long* a4 = &(u.l_data);  
    0x565561d4 <+55>:    lea     0x8(%esp),%eax  
    0x565561d8 <+59>:    mov     %eax,0x18(%esp)
```

由此可见，u.c_data、u.s_data、u.i_data和u.l_data，具有相同的起始地址。联合体的成员变量之间，是相互覆盖的关系



五、数据的对齐



华中科技大学

- CPU访问主存，若每次最多读写64位，即一次可以同时读写8个字节。例如第0-7字节可同时读写，第8-15字节可同时读写。
- 因此，一条指令中的操作数的地址按8对齐，则访问该操作数就只需要一次内存访问；否则可能需要对此操作数进行多次内存访问。例如操作数在6、7、8、9字节中，就需要进行两次内存访问。
- 为提高程序的执行效率，操作系统和编译器，通常按对齐方式分配和管理内存。例如子程序中分配局部变量的空间。

